User Management (MS1) : interactions et contrats d'API

Contexte technique

Le microservice User Management (MS1) est développé avec Spring Boot. Il gère lui-même l'inscription, l'authentification, les profils, le rôle et l'émission des JWT. L'API Gateway utilise Spring Cloud Gateway ; Keycloak n'est pas utilisé.

Le modèle de données suit le diagramme de classes : `User`, `RefreshToken`, `PasswordResetToken`, avec les énumérations `Role` (STUDENT, ADMIN) et `UserStatus` (ACTIVE, INACTIVE, SUSPENDED).

Le claim `sub` du JWT contient `User.id`. Chaque microservice stocke cet identifiant dans le type de son choix (Long, String, INT), car chaque service possède son modèle et sa base ; la seule exigence est que la valeur soit convertie sans perte.

1. Interactions avec les autres microservices

Principe général. Chaque microservice possède sa propre base de données. Aucun service ne lit directement `user_db`. Les échanges passent par des API REST ou par RabbitMQ (exchange de type topic `freespace.events`, clés de routage `user.*`). Les autres services valident les JWT localement à l'aide de la clé publique de MS1 : ils n'appellent pas MS1 pour authentifier chaque requête.

1.1 Vue d'ensemble des échanges

Émetteur → destinataire	Type d'échange	Contenu	But
React → Gateway → MS1	REST synchrone	Inscription, connexion, renouvellement du token, déconnexion, profil, mot de passe	Créer et gérer l'identité de l'étudiant
Gateway et MS2 à MS7	JWT vérifié localement	Signature, expiration, claims `sub` et `role`	Authentifier les appels sans consulter `user_db`
MS3 Lost & Found → MS1	REST interne synchrone	`GET /internal/users/{userId}`	Récupérer le nom et le prénom de l'étudiant pour enregistrer `nomPublieur` ou `nomExpediteur`
MS5 Study Community → MS1	REST interne synchrone	`GET /internal/users/{userId}`	Obtenir le profil minimal (pseudo, photo) d'un membre de groupe
MS1 → MS7	RabbitMQ asynchrone	`UserRegistered`, `PasswordResetRequested`, `UserProfileUpdated`	Message de bienvenue, e-mail de réinitialisation, copie locale du pseudo et de la photo pour la messagerie
MS1 → MS4, MS5, MS7	RabbitMQ asynchrone	`UserAccountStatusChanged`	Réagir à une suspension, une désactivation ou une réactivation

1.2 Rôle de chaque service vis-à-vis de MS1

Service	Ce qu'il fait avec MS1	Mode
React / Gateway	Envoie les appels utilisateur vers `/api/users/**` et `/api/admin/users/**` ; les endpoints `/internal/**` ne sont pas exposés par la Gateway. Les routes publiques (register, login, refresh, password/forgot, password/reset) passent sans JWT ; toutes les autres exigent un Bearer token valide.	REST synchrone + JWT
MS2 Study Space	Aucun échange direct avec MS1. Il vérifie le JWT de l'étudiant et utilise `sub` comme `studentId` d'une `Reservation`.	JWT local
MS3 Lost & Found	Utilise `sub` du JWT (et non un identifiant envoyé par le client) pour appeler `GET /internal/users/{userId}` et obtenir le nom et le prénom, copiés dans `Annonce.nomPublieur` ou `Message.nomExpediteur`. Il ne stocke pas l'identifiant et ne consomme aucun événement : le statut du compte est vérifié à chaque publication ou envoi de message.	JWT + REST
MS4 Student Expression	Utilise `sub` comme `auteurId` (publication, commentaire, signalement) ou `utilisateurId` (réaction) ; ne conserve jamais l'e-mail ni le nom. Consomme `UserAccountStatusChanged` : le `statut` des publications d'un compte suspendu passe à « masquée ».	JWT + événement
MS5 Study Community	Utilise `sub` comme `createurId`, `etudiantId`, `auteurId` ou `organisateurId`. Appelle `GET /internal/users/{userId}` pour afficher le pseudo et la photo d'un membre (non stockés dans MS5), et consomme `UserAccountStatusChanged` : les adhésions d'un compte suspendu passent à `StatutAdhesion.EXCLUE`.	JWT + REST + événement
MS6 Campus Issues	Utilise `sub` comme `userId` d'un problème (Issue), `authorId` d'un commentaire et `changedBy` d'un historique. Les actions réservées aux administrateurs s'appuient sur le claim `role` du JWT ; MS6 ne consomme aucun événement.	JWT local
MS7 Notification & Messaging	Consomme `UserRegistered`, `PasswordResetRequested`, `UserProfileUpdated` et `UserAccountStatusChanged` ; il n'appelle jamais MS1 directement.	Événements

1.3 Exemple synchrone : publication d'une annonce dans MS3 Lost & Found

Pour enregistrer une annonce, MS3 doit connaître le nom et le prénom du publieur (attribut `nomPublieur`). Il ne peut pas continuer sans la réponse de MS1 : c'est le cas typique d'un appel synchrone.

1. L'étudiant connecté remplit le formulaire et l'envoie à MS3 avec son JWT.

2. MS3 vérifie la signature du token et lit `sub`.
3. MS3 appelle `GET /internal/users/{userId}` avec un jeton de service.
4. MS1 contrôle l'identité du service appelant, lit `user_db` et renvoie les champs autorisés (nom, prénom, statut).
5. Si le statut est ACTIVE, MS3 enregistre l'annonce avec `nomPublieur` dans sa base PostgreSQL et répond 201 ; sinon il refuse (403).

Si MS1 ne répond pas, MS3 refuse temporairement la demande plutôt que d'enregistrer une annonce sans identité vérifiée. Le même appel sert à renseigner `nomExpediteur` lors de l'envoi d'un message.

1.4 Exemple asynchrone : suspension d'un compte

Quand un administrateur suspend un compte, plusieurs services doivent réagir, mais MS1 n'a pas à les connaître ni à attendre leur réponse : il publie un fait et continue.

1. L'administrateur envoie `PATCH /api/admin/users/{userId}/status` avec le statut SUSPENDED.
2. MS1 met à jour `User.status` et révoque les refresh tokens du compte (`RefreshToken.revoke()`).
3. MS1 répond 200 à l'administrateur.
4. MS1 publie `UserAccountStatusChanged` (clé `user.status.changed`) dans RabbitMQ.
5. Chaque consommateur réagit de son côté : MS4 masque les publications, MS5 passe les adhésions de l'étudiant à `EXCLUE` , MS7 prévient l'étudiant. MS3 n'a pas à réagir : il refusera toute nouvelle publication de ce compte au prochain appel à MS1 (étape 5 de 1.3).

Second exemple plus simple : à l'inscription, MS1 publie `UserRegistered` après avoir créé le compte, et répond 201 à React sans attendre que MS7 envoie le message de bienvenue.

1.5 Événements publiés par MS1

Routing key	Événement	Contenu principal	Consommateurs et réaction
`user.registered`	`UserRegistered`	userId, email	MS7 : message de bienvenue
`user.profile.updated`	`UserProfileUpdated`	userId, pseudo, profilePicture	MS7 : mettre à jour la copie locale du profil (messagerie)
`user.status.changed`	`UserAccountStatusChanged`	userId, status (ACTIVE, INACTIVE ou SUSPENDED)	MS4 : masquer les publications. MS5 : passer les adhésions à `EXCLUE` . MS7 : prévenir l'étudiant
`user.password.reset.requested`	`PasswordResetRequested`	userId, email, resetToken, expiresAt	MS7 : envoyer le lien ou le code de réinitialisation

Règles d'échange. MS1 ne consomme aucun événement : il est la source de l'identité et ne dépend d'aucun autre microservice. Chaque événement porte un `eventId` unique et une date `occurredAt` ; les consommateurs doivent être idempotents, car un même message peut être livré deux fois. Ne jamais publier un mot de passe, son hash, un JWT ou un refresh token. Le `resetToken` est le seul jeton transmis : il est à usage unique et de courte durée (`expiresAt`), car MS7 en a besoin pour construire le lien de réinitialisation.

2. Contrats d'API du microservice User Management

Les chemins ci-dessous sont les chemins publics exposés par Spring Cloud Gateway. En développement, MS1 peut écouter directement sur `http://localhost:8081` . Les corps de requête et de réponse sont à figer dans OpenAPI/Swagger.

2.1 Endpoints publics

Méthode	Endpoint	Fonction	Codes principaux
POST	`/api/users/register`	Créer un compte (rôle STUDENT, statut ACTIVE)	201, 400, 409
POST	`/api/users/login`	Vérifier les identifiants et émettre les tokens	200, 400, 401, 423, 429
POST	`/api/users/refresh`	Renouveler l'access token	200, 401
POST	`/api/users/logout`	Révoquer le refresh token (`revoked = true`)	204, 401
GET	`/api/users/me`	Consulter le profil connecté	200, 401
PUT	`/api/users/me`	Modifier le profil connecté (firstName, lastName, pseudo, profilePicture, bio)	200, 400, 401, 409
PATCH	`/api/users/me/password`	Changer le mot de passe	204, 400, 401
POST	`/api/users/password/forgot`	Demander une réinitialisation (crée un `PasswordResetToken`)	202, 400
POST	`/api/users/password/reset`	Valider le token et définir un nouveau mot de passe	204, 400, 410

Codes du login. 401 : identifiants invalides. 423 : compte non ACTIVE (INACTIVE ou SUSPENDED), d'après `User.status` . 429 : limitation de débit appliquée par la Gateway, avant d'atteindre MS1.

2.2 Endpoints internes et administration

Méthode	Endpoint	Fonction	Codes principaux
GET	`/internal/users/{userId}`	Lire un profil minimal (id, firstName, lastName, pseudo, profilePicture, status) pour un service autorisé	200, 401, 403, 404
GET	`/internal/users/{userId}/status`	Lire le statut du compte	200, 401, 403, 404
PATCH	`/api/admin/users/{userId}/role`	Modifier le rôle (admin) ; le nouveau rôle apparaît dans le claim `role` au prochain	200, 400, 401,

		renouvellement du token	403, 404
PATCH	`/api/admin/users/{userId}/status`	Suspendre, désactiver ou réactiver (admin)	200, 400, 401, 403, 404

Les endpoints `/internal/**` doivent être protégés par une authentification service-à-service (client credentials ou mTLS, par exemple). Les endpoints `/api/admin/**` exigent le rôle ADMIN.

3. Choix de la base de données

MS1 utilise **PostgreSQL**, dans une base dédiée `user_db` (une base par microservice, comme MS3 Lost & Found). Aucun autre service ne s'y connecte : ils passent par l'API REST ou par les événements.

Pourquoi PostgreSQL :

- **Données relationnelles.** `User` possède des `RefreshToken` et des `PasswordResetToken` (compositions 1 — 0..*) : clés étrangères et suppression en cascade garantissent qu'aucun token ne survit à son compte.
- **Transactions ACID.** Une suspension met à jour `User.status` et révoque les refresh tokens dans la même transaction ; une réinitialisation marque le token comme utilisé et change le mot de passe en une seule opération, sans état intermédiaire.
- **Contraintes d'unicité.** `email`, `pseudo` et `token` sont uniques au niveau de la base, ce qui fonde le code 409 à l'inscription et protège contre les doublons en cas de requêtes simultanées. Des index sur `token` et `expiresAt` accélèrent la validation et le nettoyage des tokens expirés.
- **Cohérence avec la stack.** Spring Data JPA/Hibernate gère nativement PostgreSQL, avec des migrations versionnées (Flyway ou Liquibase). MS3 utilise déjà PostgreSQL : un seul SGBD à installer, sauvegarder et maîtriser.
- **Pas de besoin d'une base NoSQL.** Le schéma est stable, le volume est modeste (les étudiants d'un campus) et les accès sont simples. Les JWT étant validés localement par les autres services, MS1 n'est pas sollicité à chaque requête, donc pas de cache dédié nécessaire.